
Intro Deep Learning Session 1

by adsoft

What is Deep Learning?

Some of the most impressive advances in artificial intelligence in recent years have been in the field of *deep learning*. *Natural language translation, image recognition, and game playing* are all tasks where deep learning models have neared or even exceeded human-level performance.

What is a tensor

A tensor is the basic building block of modern machine learning.

At its core it's a data container.

Mostly it contains numbers.
Sometimes it even contains strings,
but that's rare. So think of it as a
bucket of numbers

There are multiple sizes of tensors. Let's
go through the most basic ones that
you'll run across in deep learning, which
will be between 0 and 5 dimensions

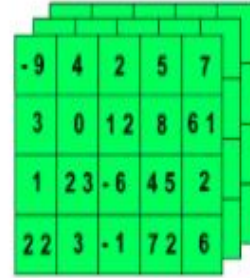
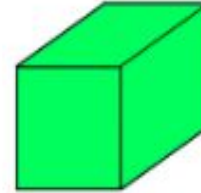
1D TENSOR /
VECTOR



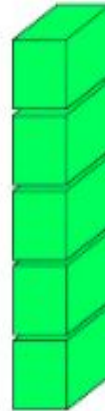
2D TENSOR /
MATRIX

-9	4	2	5	7
3	0	12	8	61
1	23	-6	45	2
22	3	-1	72	6

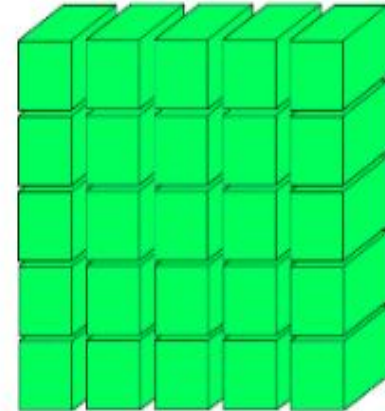
3D TENSOR /
CUBE



-9	4	2	5	7
3	0	12	8	61
1	23	-6	45	2
22	3	-1	72	6



4D TENSOR
VECTOR OF CUBES



5D TENSOR
MATRIX OF CUBES

<https://www.youtube.com/watch?v=bPPLCrjQC>
[BQ](#)

Deep learning is an approach to machine learning characterized by **deep stacks** of computations.

This depth of computation is what has enabled deep learning models to disentangle the kinds of complex and hierarchical patterns found in the most challenging real-world datasets



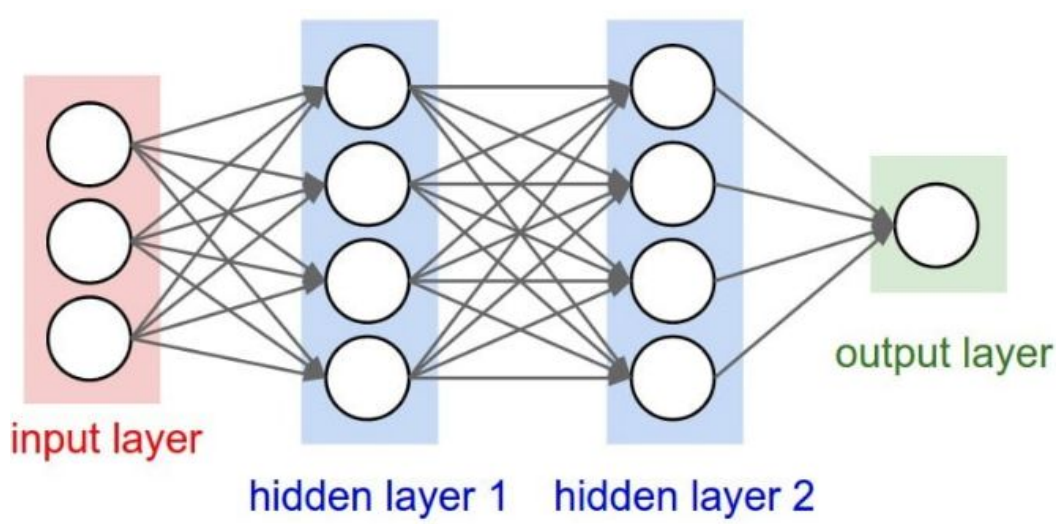
Deep Learning

Using Python and Tensorflow you'll learn how to:

- create a **fully-connected** neural network architecture
- apply neural nets to two classic ML problems: **regression** and **classification**
- train neural nets with **stochastic gradient descent**, and
- improve performance with **dropout**, **batch normalization**, and other techniques

A Single Neuron

Learn about **linear units**, the building blocks of deep learning.



Two or more hidden layers comprise a Deep Neural Network

The Linear Unit

The input is x . Its connection to the neuron has a **weight** which is w .

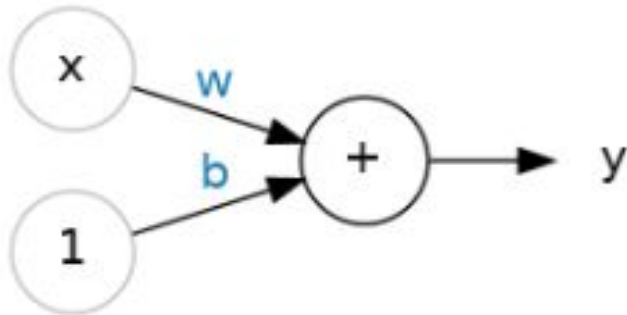
Whenever a value flows through a connection, you multiply the value by the connection's weight. For the input x , what reaches the neuron

is $w * x$. ***A neural network "learns" by modifying its weights.***

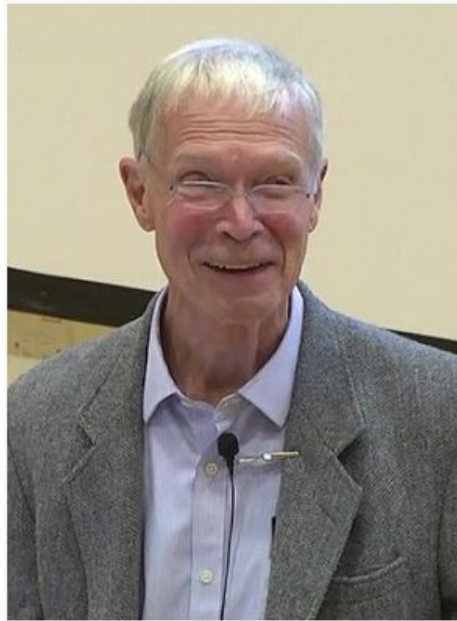
The b is a special kind of weight we call the **bias**. The bias doesn't have any input data associated with it; instead, we put a 1 in the diagram so that the value that reaches the neuron is just b (since $1 * b = b$). The bias enables the neuron to modify the output independently of its inputs.

The y is the value the neuron ultimately outputs. To get the output, the neuron sums up all the values it receives through its connections. This neuron's activation is $y = w * x + b$, or as a formula

$$y = wx + b$$



The Linear Unit: $y = wx + b$



Almacenar un conjunto de p patrones de forma tal que cuando se presente un nuevo patrón, la red responda produciendo alguno de los patrones previamente almacenados que más se parezca al presentado.

John Joseph Hopfield (Chicago, Illinois; 15 de julio de 1933), citado como **John Hopfield**, es un físico estadounidense reconocido por el desarrollo de la **red de Hopfield**. En el 2024 fue galardonado con el **Premio Nobel de Física** por su trabajo sobre **redes neuronales artificiales** junto con **Geoffrey Everest Hinton**.¹

Red HopField

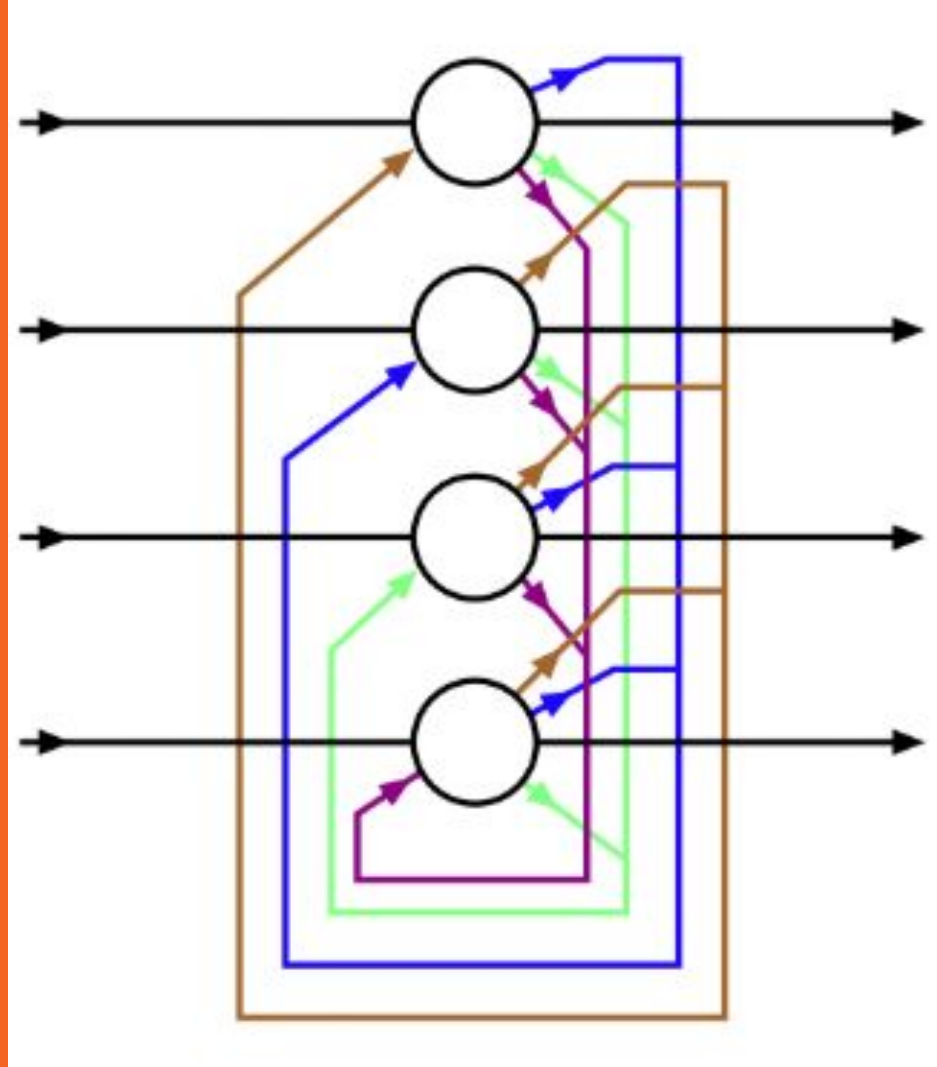
Suponiendo K clases a almacenar en una red

1. Se calculan los pesos W_{ij}

2. Se muestra a la red un vector

3. La red itera en pasos discretos hasta la convergencia.

4. El vector mostrado es el vector de salida.



Red HopField

$$\begin{aligned} \mathbf{X}_1 &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ & & & \end{bmatrix} & \mathbf{X}_2 &= \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ & & & \end{bmatrix} \end{aligned}$$

$$\begin{aligned} \mathbf{X}_1 &= (1, 1, 1, -1) \\ \mathbf{X}_2 &= (-1, -1, -1, 1) \end{aligned}$$

$$\mathbf{X}_1^T \cdot \mathbf{X}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{X}_2^T \cdot \mathbf{X}_2 = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & -1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix}$$



$$\mathbf{X}_1^T \cdot \mathbf{X}_1 + \mathbf{X}_2^T \cdot \mathbf{X}_2 = \begin{bmatrix} 2 & 2 & 2 & -2 \\ 2 & 2 & 2 & -2 \\ 2 & 2 & 2 & -2 \\ -2 & -2 & -2 & 2 \end{bmatrix},$$

haciendo la diagonal = 0:

$$\mathbf{T} = \begin{bmatrix} 0 & 2 & 2 & -2 \\ 2 & 0 & 2 & -2 \\ 2 & 2 & 0 & -2 \\ -2 & -2 & -2 & 0 \end{bmatrix}$$

Red HopField

Supongamos que deseamos recuperar el patrón mas cercano a:

$$\mathbf{A} = (1 \ 1 \ 1 \ -1)$$

1º. Paso: $\mathbf{U(0)} = \mathbf{A}$

2º. Paso: $\mathbf{U(0)} \cdot \mathbf{T} = (1 \ 1 \ 1 \ -1) \begin{pmatrix} 0 & 2 & 2 & -2 \\ 2 & 0 & 2 & -2 \\ 2 & 2 & 0 & -2 \\ -2 & -2 & -2 & 0 \end{pmatrix} = (6 \ 6 \ 6 \ -6)$

$$\mathbf{U(1)} = \mathbf{F(U(0) \cdot T)} = (1 \ 1 \ 1 \ -1)$$

En este punto se cumple que $\mathbf{U(1)}$ es igual al $\mathbf{U(0)}$, por lo que el sistema ya está estable y el proceso termina.

El patrón mas parecido a $\mathbf{A}$ es $(1 \ 1 \ 1 \ -1)$

donde F es una función escalón definida como:

$$F(x) = \begin{cases} 1 & \text{si } x > 0 \\ -1 & \text{si } x < 0 \\ U_j(t) & \text{si } x = 0 \text{ (sin cambio)} \end{cases}$$

Red HopField

Ahora hallaremos el patrón mas parecido a $\mathbf{A} = (-1 \ -1 \ -1 \ -1)$

$$\mathbf{U}(0) \cdot \mathbf{T} = (-1 \ -1 \ -1 \ -1) \begin{pmatrix} 0 & 2 & 2 & -2 \\ 2 & 0 & 2 & -2 \\ 2 & 2 & 0 & -2 \\ -2 & -2 & -2 & 0 \end{pmatrix} = (-2 \ -2 \ -2 \ 6)$$

$$\mathbf{U}(1) = \mathbf{F}(\mathbf{U}(0) \cdot \mathbf{T}) = (-1 \ -1 \ -1 \ 1) \quad \mathbf{U}(1) \neq \mathbf{U}(0)!$$

$$\mathbf{U}(1) \cdot \mathbf{T} = (-1 \ -1 \ -1 \ 1) \begin{pmatrix} 0 & 2 & 2 & -2 \\ 2 & 0 & 2 & -2 \\ 2 & 2 & 0 & -2 \\ -2 & -2 & -2 & 0 \end{pmatrix} = (-6 \ -6 \ -6 \ 6)$$

$$\mathbf{U}(2) = (-1 \ -1 \ -1 \ 1) \quad \mathbf{U}(2) = \mathbf{U}(1) \Rightarrow \text{FIN!}$$

donde F es una función escalón definida como:

$$F(x) = \begin{cases} 1 & \text{si } x > 0 \\ -1 & \text{si } x < 0 \\ U_j(t) & \text{si } x = 0 \text{ (sin cambio)} \end{cases}$$

Read more

https://www.freecodecamp-org.translate.google.com/news/linear-algebra-roadmap/?x_tr_sl=en&x_tr_tl=es&x_tr_hl=es&x_tr_pto=tc



Challenge

Programar en Java, C, Python, etc. la red Hopfield del ejemplo anterior, solo usando arrays, ciclos for, while y sentencias if, else.

**NOTA: no usar modulos numpy.
Tensorflow, pytorch, etc.**